Title:

Implementation of MLP, Sparse Autoencoder, and Restricted Boltzmann Machine from Scratch

Name: Manya
Course: Deep Learning
Submission: 24-02-2026

Introduction

In this project, three neural network models were implemented completely from scratch using NumPy:

- Multi-Layer Perceptron (MLP)
- Sparse Autoencoder
- Restricted Boltzmann Machine (RBM)

The objective was to understand internal learning mechanisms, weight updates, and unsupervised feature learning without using deep learning libraries such as PyTorch or TensorFlow.

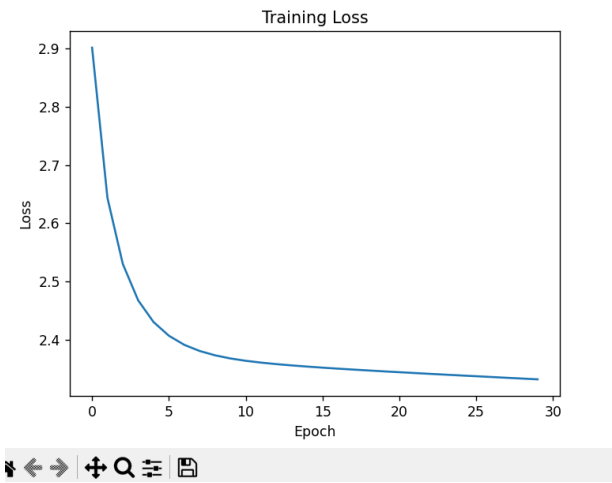
Model 1 – Multi-Layer Perceptron (MLP)

Architecture

- Input Layer: 784 neurons (28×28 image flattened)
-
- Hidden Layer 1: 128 neurons (ReLU activation)
- Hidden Layer 2: 64 neurons (ReLU activation)
- Output Layer: 10 neurons (Softmax)

Hyperparameters

- Learning Rate: 0.01
- Epochs: 30
- Loss Function: Cross-Entropy
- Optimizer: Gradient Descent



Training Curve

The training loss decreased steadily over epochs, indicating correct backpropagation and learning of patterns in the dataset.

Model 2 – Sparse Autoencoder

Architecture

- Input: 784 neurons
- Encoder: 128 → 64
- Bottleneck: 32 neurons (undercomplete)
- Decoder: 64 → 128 → 784
- Activation: Sigmoid

Sparsity Constraint

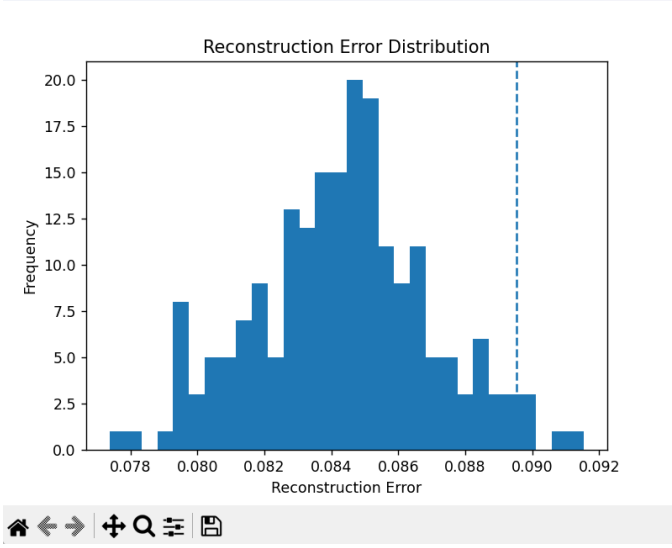
L1 regularization was added to the bottleneck layer:

Total Loss = MSE + λ × |latent activations|

Where:

- MSE = reconstruction loss
- λ = 0.001

This encourages sparse feature learning.



Outlier Detection Method

Outliers were detected using reconstruction error:

- Compute reconstruction error per sample
- Threshold = Mean + 2 × Standard Deviation
- Samples above threshold classified as anomalies

Explain:

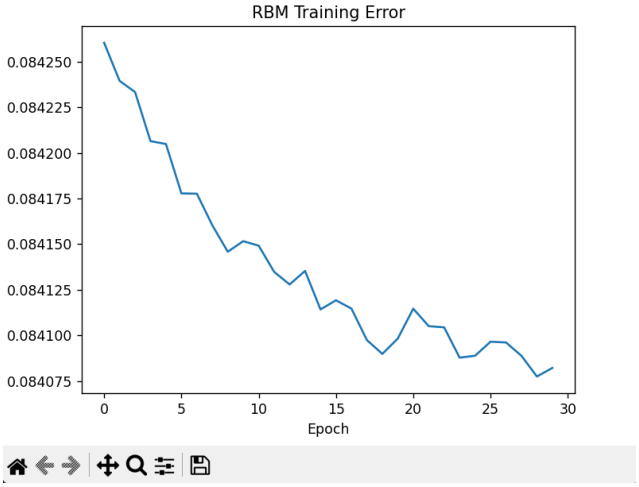
Most data points cluster around low reconstruction error. Outliers appear in the high-error tail beyond the threshold.

Model 3 – Restricted Boltzmann Machine (RBM)

Architecture

- Visible Layer: 784 units
- Hidden Layer: 64 units
- Weight matrix initialized randomly

- Binary stochastic sampling



Training Method

RBM was trained using:

Contrastive Divergence (CD-1)

Steps:

1. Positive phase: compute hidden probabilities
2. Negative phase: reconstruct visible units
3. Update weights using difference

Hyperparameters

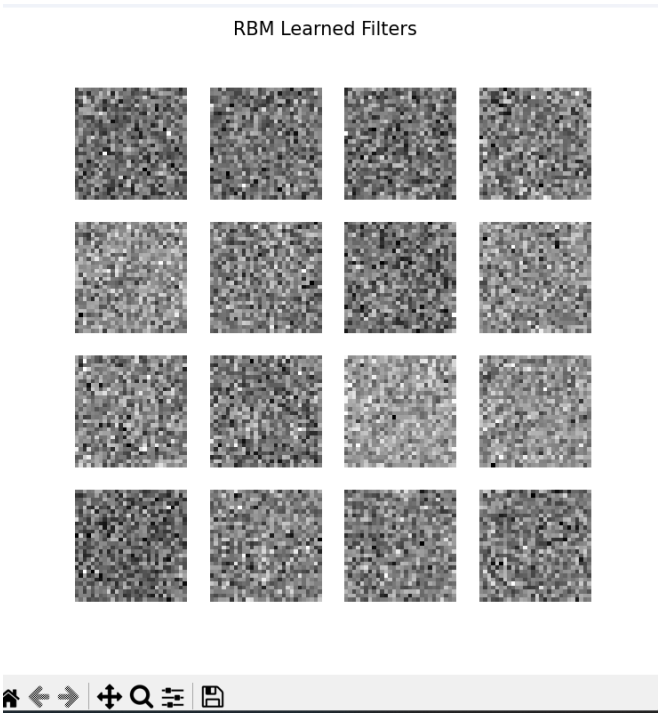
- Learning Rate: 0.01
- Epochs: 30

Training Curve

Explain:

Reconstruction error decreased over epochs, showing the model learned meaningful features.

Filter Visualization



Explain:

Filters resemble basic stroke patterns and localized pixel activations, indicating feature extraction capability.

Comparison: Autoencoder vs RBM

Autoencoder:

- Deterministic
- Trained using backpropagation
- Learns compressed representations
- Good for dimensionality reduction and anomaly detection

RBM:

- Probabilistic
- Trained using Contrastive Divergence
- Models data distribution
- Useful for generative modeling

Evaluation & Performance

MLP achieved stable training with decreasing cross-entropy loss.

Sparse Autoencoder successfully:

- Compressed input data
- Identified outliers using reconstruction error

RBM:

- Learned meaningful feature filters
- Showed consistent reduction in reconstruction error

All models were implemented from scratch, validating theoretical understanding and correctness.

Reproducibility

To run the project:

1. Install dependencies:
    - numpy

    - matplotlib
    - pandas

2. Run MLP:
   python train_mlp.py
3. Run Autoencoder:
   python train_autoencoder.py
4. Run RBM:
   python train_rbm.py

All plots are generated automatically during execution.

Conclusion

This project demonstrated practical implementation of supervised and unsupervised neural networks from scratch. Sparse representations and probabilistic modeling techniques were successfully applied. The results validate correct mathematical implementation and learning behavior.